

Machine Learning Models

Shaon Khan

Simple Linear Regression

১. মূল সমীকরণ (The Core Formula)

সিম্পল লিনিয়ার রিগ্রেশন মূলত একটি ইনপুট ফিচার (X) এবং একটি আউটপুট টার্গেটের (y) মধ্যে সরলরৈখিক সম্পর্ক তৈরি করে। এর গাণিতিক সূত্রটি হলো:

$$\hat{y} = wX + b$$

$\hat{y}$ (y-hat): মডেলের প্রেডিক্ট করা মান (Predicted Value)

X: ইনপুট ফিচার (Independent Variable)

w (Weight/Slope): রেখাটির ঢাল। X এক ইউনিট বাড়লে y কতটুকু বাড়বে বা কমবে, তা w নির্ধারণ করে

b (Bias/Intercept): ইন্টারসেপ্ট। X-এর মান 0 হলে y-এর মান কত হবে, তা এটি নির্দেশ করে

২. কস্ট ফাংশন (Cost Function)

মডেলের প্রেডিকশন আসল ডেটার তুলনায় কতটা ভুল করছে, তা পরিমাপ করার জন্য Mean Squared Error (MSE) কস্ট ফাংশন ব্যবহার করা হয়। এর গাণিতিক রূপ:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

যেহেতু

$$\hat{y}_i = wX_i + b$$

তাই সমীকরণটি দাঁড়ায়:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (wX_i + b - y_i)^2$$

n: মোট ডেটা স্যাম্পলের সংখ্যা

y_i : আসল মান (Actual Value)

$\hat{y}_i$: মডেলের প্রেডিক্ট করা মান

৩. কস্ট ফাংশনের ডেরিভেটিভ (Partial Derivatives)

গ্রাডিয়েন্ট ডিসেন্ট পদ্ধতিতে কস্ট ফাংশন (J) এর মান সর্বনিম্ন করার জন্য আমাদের জানতে হবে w এবং b পরিবর্তনের সাথে সাথে কস্ট কতটা পরিবর্তিত হচ্ছে। এর জন্য ক্যালকুলাসের চেইন রুল (Chain Rule) ব্যবহার করে আংশিক অন্তরীকরণ (Partial Derivative) করা হয়:

w-এর সাপেক্ষে ডেরিভেটিভ (dJ/dw):

$$\frac{\partial J}{\partial w} = \frac{2}{n} \sum_{i=1}^n (w X_i + b - y_i) \cdot X_i$$

b-এর সাপেক্ষে ডেরিভেটিভ (dJ/db):

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^n (w X_i + b - y_i)$$

(এখানে $wX_i + b - y_i$ অংশটি হলো মূলত প্রেডিকশন এবং আসল মানের মধ্যকার ভুল বা Error)

৪. প্যারামিটার আপডেট সূত্র (Weight & Bias Update)

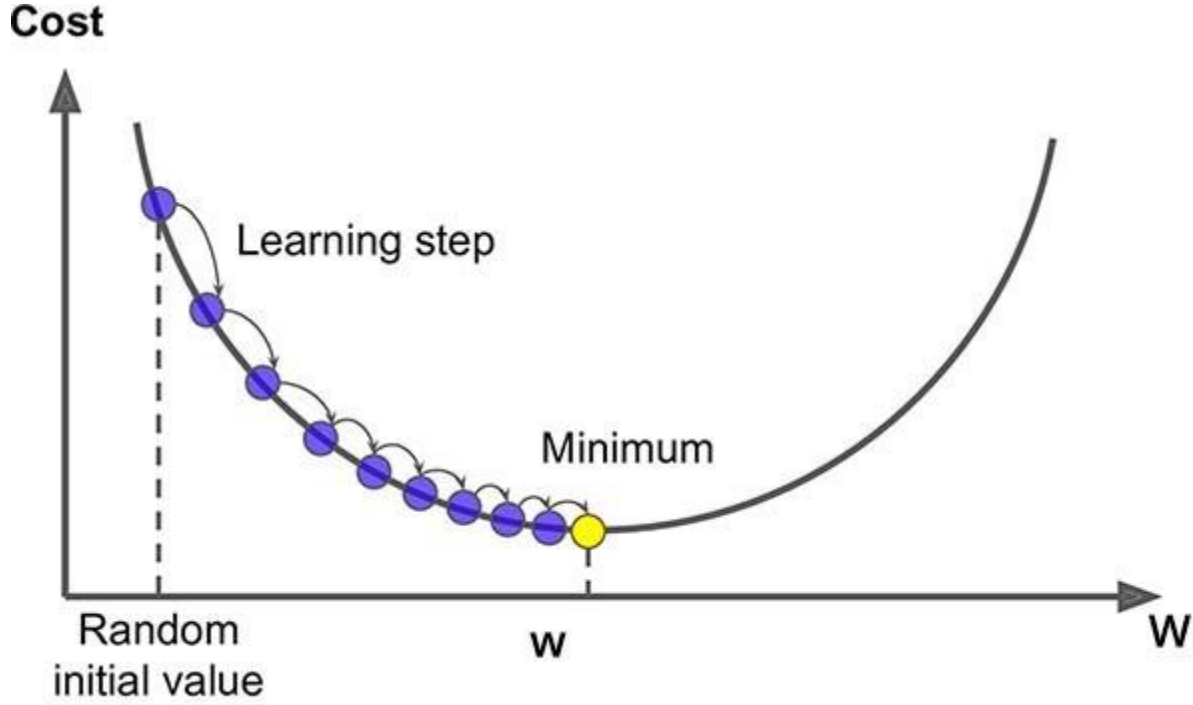
প্রতিটি ইটারেশনে বা লুপে প্রাপ্ত গ্রাডিয়েন্ট বা ডেরিভেটিভের ওপর ভিত্তি করে লার্নিং রেট (α) গুণ করে w এবং b-কে আপডেট করা হয়, যাতে তারা আস্তে আস্তে সঠিক মানের দিকে এগোতে পারে:

$$w = w - \alpha \cdot \frac{\partial J}{\partial w}$$
$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

α (Alpha): লার্নিং রেট (Learning Rate)। এটি নির্ধারণ করে মডেল কতটা বড় বা ছোট স্টেপ ফেলে গ্লোবাল মিনিমামের দিকে নামবে

৫. কস্ট ফাংশন গ্রাফ (Cost Function Graph)

লিনিয়ার রিগ্রেশনের কস্ট ফাংশনের গ্রাফটি দেখতে একটি বাটি বা বোলের (Bowl) মতো হয়, যাকে গাণিতিক ভাষায় Convex Function বলা হয়। গ্রাডিয়েন্ট ডিসেন্ট পাহাড়ের চূড়া থেকে আস্তে আস্তে পা ফেলে বাটির একদম তলানিতে পৌঁছানোর চেষ্টা করে, যেখানে ভুলের পরিমাণ সবচেয়ে কম।



ব্যাকএন্ডে কীভাবে কাজ করে? (Full Backend Workflow)

ধরা যাক, আমাদের কাছে একটি ডেটাসেট আছে। ব্যাকএন্ডে পুরো প্রসেসটি নিচের ধাপে ধাপে সম্পন্ন হয়:

- Initialization: শুরুতেই কম্পিউটার আন্দাজে $w = 0$ এবং $b = 0$ ধরে নেয়
- Forward Pass (Prediction): এই প্রাথমিক w এবং b দিয়ে পুরো ডেটাসেটের জন্য $\hat{y} = wX + b$ সূত্র ব্যবহার করে প্রেডিকশন হিসাব করা হয়
- Compute Cost: প্রেডিকশন বের করার পর আসল মানের সাথে তুলনা করে MSE সূত্রের মাধ্যমে বর্তমান কস্ট বা ভুলের পরিমাণ (J) মাপা হয়
- Backward Pass (Gradient Computation): আংশিক অন্তরীকরণের সূত্রের মাধ্যমে বর্তমান ভুলের জন্য w এবং b কতটা দায়ী (অর্থাৎ গ্রাডিয়েন্ট $\partial J/\partial w$ এবং $\partial J/\partial b$) তা ক্যালকুলেট করা হয়
- Update Parameters: এই গ্রাডিয়েন্ট ব্যবহার করে w এবং b -এর পুরানো মান থেকে লার্নিং রেট গুণ করে নতুন মান আপডেট করা হয়
- Iteration: এই ২ থেকে ৫ নম্বর ধাপটি লুপের মাধ্যমে হাজার হাজার বার (যেমন 1,500 বার) চলতে থাকে। প্রতিবারে বাট্টির তলানির দিকে নামতে নামতে কস্ট কমতে থাকে এবং একপর্যায়ে স্থির হয়ে যায়

From Scratch (No Scikit-Learn)

```
1 import numpy as np
2
3 def make_prediction(X, w, b):
4     return w * X + b
5
6 def cost_function(X, y, w, b):
7     n = len(y)
8     y_pred = make_prediction(X, w, b)
9     return (1 / n) * np.sum((y_pred - y) ** 2)
10
11 def calculate_gradient(X, y, w, b):
12     n = len(y)
13     y_pred = make_prediction(X, w, b)
14     dw = (2 / n) * np.sum((y_pred - y) * X)
15     db = (2 / n) * np.sum((y_pred - y))
16     return dw, db
17
18 def gradient_descent(X, y, w, b, lr, epochs):
19     for i in range(epochs):
20         dw, db = calculate_gradient(X, y, w, b)
21         w = w - lr * dw
22         b = b - lr * db
23         if i % 100 == 0:
24             print("epoch:", i, "cost:", cost_function(X, y, w, b))
25     return w, b
26
27 X = np.array([1, 2, 3, 4, 5])
28 y = np.array([2, 4, 6, 8, 10])
29
30 w, b = gradient_descent(X, y, 0, 0, 0.01, 1000)
31
32 print("Final w:", w)
33 print("Final b:", b)
```

Linear Regression using Scikit Learn

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression, SGDRegressor
3 from sklearn.preprocessing import StandardScaler
4
5 X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
6 y = np.array([2, 4, 6, 8, 10])
7
8 lr_model = LinearRegression()
9 lr_model.fit(X, y)
10
11 print("Linear Regression")
12 print("w:", lr_model.coef_[0])
13 print("b:", lr_model.intercept_)
14
15 print("Pred:", lr_model.predict(X))
```

SDG Regression using Scikit Learn

```
18 scaler = StandardScaler()
19 X_scaled = scaler.fit_transform(X)
20
21 sgd_model = SGDRegressor(
22     learning_rate="constant",
23     eta0=0.01,
24     max_iter=1000,
25     tol=1e-3
26 )
27
28 sgd_model.fit(X_scaled, y)
29
30 print("\nSGD Regression")
31 print("w:", sgd_model.coef_[0])
32 print("b:", sgd_model.intercept_[0])
33 print("Pred:", sgd_model.predict(X_scaled))
```

Multiple Linear Regression

মাল্টিপল লিনিয়ার রিগ্রেশন (Multiple Linear Regression)

মাল্টিপল লিনিয়ার রিগ্রেশন মূলত সিম্পল লিনিয়ার রিগ্রেশনেরই সম্প্রসারিত রূপ, যেখানে একাধিক ইনপুট ফিচার $X_1, X_2, \dots, X_m$ ব্যবহার করা হয়।

১. মূল সমীকরণ (The Core Formula)

যখন একাধিক ফিচার থাকে, তখন প্রেডিকশন হয়:

$$\hat{y} = w_1X_1 + w_2X_2 + \dots + w_mX_m + b$$

এটিকে ভেক্টরাইজড ফর্মে লেখা হয়:

$$\hat{y} = XW + b$$

যেখানে:

- X : ইনপুট ফিচার ম্যাট্রিক্স (সাইজ $n \times m$)
- W : ওয়েট ভেক্টর $[w_1, w_2, \dots, w_m]^T$
- b : বায়াস (স্কেলার)
- $\hat{y}$: প্রেডিক্টেড আউটপুট

২. কস্ট ফাংশন (Cost Function)

Mean Squared Error (MSE):

$$J(W, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

ভেক্টর ফর্মে:

$$J(W, b) = \frac{1}{n} \|XW + b - y\|^2$$

৩. কস্ট ফাংশনের ডেরিভেটিভ (Partial Derivatives)

ওয়েটের জন্য গ্রাডিয়েন্ট:

$$\frac{\partial J}{\partial W} = \frac{2}{n} X^T (XW + b - y)$$

বায়াসের জন্য গ্রাডিয়েন্ট:

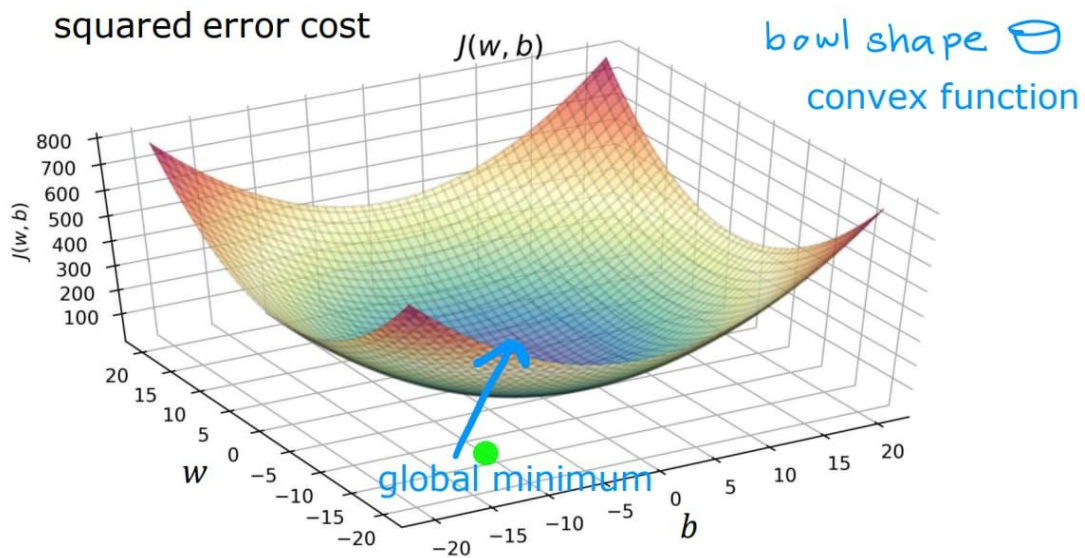
$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^n (XW + b - y)$$

৪. প্যারামিটার আপডেট (Weight & Bias Update)

$$W = W - \alpha \cdot \frac{\partial J}{\partial W}$$
$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

৫. কস্ট ফাংশন গ্রাফ (Cost Function Graph)

মাল্টিপল ফিচারের ক্ষেত্রে কস্ট ফাংশন একটি **high-dimensional convex surface** তৈরি করে।
এটি দৃশ্যমানভাবে 2D বা 3D বাটি না হলেও গাণিতিকভাবে একটি মাত্র গ্লোবাল মিনিমাম থাকে।



ব্যাকএন্ড ওয়ার্কফ্লো (Full Backend Workflow)

- Initialization: $W = 0, b = 0$
- Forward Pass:

$$\hat{y} = XW + b$$

- Compute Cost: MSE দিয়ে error হিসাব
- Backward Pass:

$$X^T \cdot error$$

- Update Parameters: W এবং b আপডেট
- Convergence: লস মিনিমাম না হওয়া পর্যন্ত লুপ

Python Code

```
1 import numpy as np
2
3 X = np.array([
4     [1.0, 2100.0],
5     [2.0, 1500.0],
6     [3.0, 2800.0],
7     [4.0, 1200.0],
8     [5.0, 3200.0]
9 ])
10
11 y = np.array([45.0, 35.0, 60.0, 28.0, 70.0])
12
13 def predict(X, W, b):
14     return np.dot(X, W) + b
15
16 def compute_cost(X, y, W, b):
17     n = len(X)
18     return (1/n) * np.sum((predict(X, W, b) - y) ** 2)
19
20 def compute_gradient(X, y, W, b):
21     n = len(X)
22     error = predict(X, W, b) - y
23     dW = (2/n) * np.dot(X.T, error)
24     db = (2/n) * np.sum(error)
25     return dW, db
```

```

27- def gradient_descent(X, y, W, b, alpha, iters):
28-     for i in range(iters):
29-         dw, db = compute_gradient(X, y, W, b)
30-
31-         W = W - alpha * dw
32-         b = b - alpha * db
33-
34-     return W, b
35-
36- X_mean = np.mean(X, axis=0)
37- X_std = np.std(X, axis=0)
38- X_scaled = (X - X_mean) / X_std
39-
40- W, b = gradient_descent(X_scaled, y, np.zeros(X.shape[1]), 0.0, 0.01, 50000)
41-
42- print(W, b)

```

Using Scikit Learn

```

1  from sklearn.linear_model import LinearRegression, SGDRegressor
2  from sklearn.preprocessing import StandardScaler
3
4  lr = LinearRegression()
5  lr.fit(X, y)
6
7  print(lr.coef_, lr.intercept_)
8
9  scaler = StandardScaler()
10 X_scaled = scaler.fit_transform(X)
11
12 sgd = SGDRegressor(max_iter=10000, eta0=0.01, random_state=42)
13 sgd.fit(X_scaled, y)
14
15 print(sgd.coef_)
16 print(sgd.predict(X_scaled[:1]))

```

লিনিয়ার রিগ্রেশনের ক্ষেত্রে StandardScaler ব্যবহার না করলেও সাধারণত সমস্যা হয় না, কারণ LinearRegression মডেলটি একটি ডাইরেক্ট ম্যাথমেটিক্যাল সলিউশন (OLS) ব্যবহার করে একবারেই ওজন (w) এবং বায়াস (b) এর সঠিক মান বের করে ফেলে। এই পদ্ধতিতে কোনো ধাপে ধাপে ইটারেশন বা লার্নিং প্রসেস নেই, বরং সরাসরি একটি সূত্র ব্যবহার করা হয়: $(X^T X)^{-1} X^T y$ । ফলে ফিচারের মান বড় বা ছোট যাই হোক না কেন, এটি গ্লোবাল মিনিমাম সঠিকভাবে বের করতে পারে এবং স্কেলিং না করলেও এর আউটপুট সাধারণত পরিবর্তিত হয় না।

অন্যদিকে SGD (Stochastic Gradient Descent) সম্পূর্ণ ভিন্নভাবে কাজ করে। এটি ধাপে ধাপে ইটারেশনের মাধ্যমে ধীরে ধীরে সঠিক মানের দিকে এগিয়ে যায়। এখানে ফিচারের স্কেল বড় হলে গ্রাডিয়েন্ট হিসাবের সময় অত্যন্ত বড় মান তৈরি হতে পারে, যার ফলে ওয়েট হঠাৎ অনেক বেশি বেড়ে গিয়ে মডেল অস্থিতিশীল হয়ে যেতে পারে বা কস্ট ফাংশন ডাইভার্জ করে NaN হয়ে যেতে পারে। এছাড়া স্কেল না করা ডেটায় কস্ট ফাংশনের সারফেস অসম বা লম্বাটে হয়ে যায়, ফলে গ্রাডিয়েন্ট ডিসেন্ট সোজা পথে নামতে না পেরে জিগজ্যাগ পথে ধীরে ধীরে কনভার্জ করে। এই কারণে SGD ব্যবহার করার আগে StandardScaler দিয়ে ফিচারগুলোকে একই স্কেলে আনা একদম গুরুত্বপূর্ণ, যাতে ট্রেনিং স্থিতিশীল, দ্রুত এবং নির্ভুল হয়।

লিনিয়ার রিগ্রেশন (LinearRegression) এবং SGD কীভাবে কাজ করে

১. Linear Regression (OLS) কীভাবে কাজ করে

লিনিয়ার রিগ্রেশন একটি **direct mathematical solution** ব্যবহার করে। এখানে মডেল কোনো লুপ চালিয়ে শেখে না। বরং পুরো ডেটাসেট একবারে দেখে একটি নির্দিষ্ট গাণিতিক সূত্র ব্যবহার করে সরাসরি সেরা ওজন (w) এবং বায়াস (b) বের করে ফেলে।

এই সূত্রটি হলো:

$$\theta = (X^T X)^{-1} X^T y$$

এখানে মডেল আসলে এমনভাবে কাজ করে যেন সে বলছে:

“আমি পুরো ডেটা একসাথে দেখে সবচেয়ে সঠিক রেখাটা একবারেই হিসাব করে ফেলছি।”

- ◆ তাই এটি দ্রুত
- ◆ তাই এটি স্থিতিশীল
- ◆ এবং ছোট-মাঝারি ডেটাসেটের জন্য খুব ভালো

২. SGD (Stochastic Gradient Descent) কীভাবে কাজ করে

SGD সম্পূর্ণ ভিন্নভাবে কাজ করে। এটি কোনো সরাসরি সূত্র ব্যবহার করে না।

বরং এটি:

- প্রথমে w এবং b র্যান্ডমভাবে নেয়
- তারপর প্রতিটি ডেটা পয়েন্ট বা ছোট ব্যাচ দেখে
- ধাপে ধাপে ভুল কমানোর চেষ্টা করে
- প্রতিবার একটু একটু করে w এবং b আপডেট করে

এটা অনেকটা এমন:

“আমি অন্ধভাবে ধাপে ধাপে হাঁটছি, ভুল হলে দিক ঠিক করছি।”

এই আপডেট হয় এইভাবে:

$$w = w - \alpha \frac{\partial J}{\partial w}$$

৩. কখন কোনটা ব্যবহার করা উচিত

Linear Regression ব্যবহার করা ভালো যখন:

- ডেটাসেট ছোট বা মাঝারি (few thousand rows পর্যন্ত)
- তুমি দ্রুত এবং exact solution চাও
- কম্পিউটেশনাল রিসোর্স সমস্যা না
- ব্যাচ ট্রেনিং দরকার নেই

উদাহরণ:

- house price prediction
- simple business forecasting
- academic projects

SGD ব্যবহার করা ভালো যখন:

- ডেটাসেট অনেক বড় (millions of rows)
- online learning দরকার (data আসতে থাকে নতুন নতুন করে)
- memory সীমিত
- deep learning বা large-scale ML system

উদাহরণ:

- recommendation system
- real-time prediction system
- large-scale industrial ML

OLS (Ordinary Least Squares) কীভাবে কাজ করে

OLS হলো Linear Regression-এর একটি **closed-form solution** যেখানে মডেল কোনো iteration ছাড়াই সরাসরি best parameters (W, b) বের করে ফেলে।

১. Model Equation

Linear Regression-এর মূল সমীকরণ:

$$\hat{y} = XW + b$$

২. Cost Function (MSE)

OLS যেটা মিনিমাইজ করে:

$$J(W, b) = \frac{1}{n} \|XW + b - y\|^2$$

এটি মূলত prediction error-এর square mean।

৩. Goal of OLS

OLS-এর লক্ষ্য:

$$\min_{W, b} J(W, b)$$

অর্থাৎ এমন W এবং b খুঁজে বের করা যাতে error সবচেয়ে কম হয়।

৪. Bias Handling (Important Step)

গণনা সহজ করার জন্য b-কে X-এর সাথে যোগ করা হয়:

$$X' = [X \ 1]$$

এখন সমীকরণ হয়:

$$\hat{y} = X'\theta$$

যেখানে:

- θ = সব parameters (W এবং b একসাথে)

৫. Closed-form Solution (Main Formula)

OLS সরাসরি এই formula ব্যবহার করে solution বের করে:

$$\theta = (X^T X)^{-1} X^T y$$

৬. Step-by-step Mechanism

Step 1: Design Matrix তৈরি

$$X \rightarrow X'$$

Step 2: Matrix Multiplication

$$X^T X$$

Step 3: Inverse নেওয়া

$$(X^T X)^{-1}$$

Step 4: Final multiplication

$$\theta = (X^T X)^{-1} X^T y$$

OLS (Ordinary Least Squares) — Example

ধরা যাক আমাদের কাছে একটি ছোট dataset আছে:

X (Study Hours)	y (Marks)
1	2
2	4
3	6

আমরা চাই relationship বের করতে:

$$\hat{y} = wX + b$$

১. Design Matrix তৈরি (Bias সহ)

OLS-এ bias যোগ করার জন্য X-কে expand করা হয়:

$$X = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix}$$

এখানে:

- প্রথম column = feature (X)
- দ্বিতীয় column = bias (1)

২. Target Vector

$$y = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

৩. OLS Formula

$$\theta = (X^T X)^{-1} X^T y$$

এখানে:

$$\theta = \begin{bmatrix} w \\ b \end{bmatrix}$$

৪. Step-by-step Calculation

Step 1: Transpose

$$X^T = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix}$$

Step 2: Multiply $X^T X$

$$X^T X = \begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}$$

Step 3: Inverse

$$\left((X^T X)^{-1} = \begin{bmatrix} 1.5 & -3 \\ -3 & 7 \end{bmatrix} \right)$$

Step 4: Multiply with $X^T y$

$$X^T y = \begin{bmatrix} 28 \\ 12 \end{bmatrix}$$

Final Result

$$\theta = \begin{bmatrix} w \\ b \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

৫. Final Model

$$\hat{y} = 2X + 0$$

LinearRegression (sklearn.linear_model.LinearRegression)

এটি OLS (Ordinary Least Squares) এর closed-form solution ব্যবহার করে, তাই এখানে regularization বা learning-related কোনো parameter নেই। টিউন করার মতো parameter খুবই কম।

Parameter	অর্থ	কখন গুরুত্বপূর্ণ
fit_intercept (default=True)	Bias term (b) যুক্ত করবে কিনা	যদি তোমার ডেটা already mean-centered হয় বা তুমি জানো রেখা origin দিয়ে যাবে, তখন False করতে পারো। সাধারণত True রাখাই উচিত।
positive (default=False)	সব coefficient-কে non-negative (≥ 0) রাখতে বাধ্য করে	কিছু physical বা economic dataset-এ যেখানে negative weight অর্থহীন, সেখানে কাজে লাগে।
n_jobs	Parallel processing-এর জন্য কত core ব্যবহার হবে	Performance/Speed সংক্রান্ত। Model quality-তে কোনো প্রভাব নেই।
copy_X (default=True)	Input X copy করবে কিনা	Memory optimization-এর জন্য। Model performance-এ প্রভাব নেই।

সবচেয়ে গুরুত্বপূর্ণ	মন্তব্য
fit_intercept	প্রায় সব ক্ষেত্রে True রাখা উচিত
positive	বিশেষ domain-specific ক্ষেত্রে দরকার
n_jobs, copy_X	Performance ও memory সংক্রান্ত

SGDRegressor (sklearn.linear_model.SGDRegressor)

এটি Gradient Descent ভিত্তিক মডেল। এখানেই আসল hyperparameter tuning হয় এবং model performance-এর উপর সবচেয়ে বেশি প্রভাব ফেলে।

Parameter	অর্থ	কখন গুরুত্বপূর্ণ
loss (default='squared_error')	কোন cost function minimize হবে	Outlier বেশি থাকলে 'huber' ব্যবহার করলে model বেশি robust হয়।
penalty (default='l2')	Regularization-এর ধরন (l1, l2, elasticnet, None)	Overfitting কমানোর জন্য অত্যন্ত গুরুত্বপূর্ণ।
alpha (default=0.0001)	Regularization strength	সবচেয়ে গুরুত্বপূর্ণ parameter। বেশি হলে underfitting, কম হলে overfitting।
l1_ratio (default=0.15)	ElasticNet-এ L1 ও L2-এর অনুপাত	penalty='elasticnet' হলে ব্যবহার হয়।
learning_rate (default='invscaling')	Learning rate schedule	Training stability নির্ধারণ করে।
eta0 (default=0.01)	Initial learning rate	খুব বড় হলে divergence, খুব ছোট হলে slow convergence।
max_iter (default=1000)	সর্বোচ্চ কত epoch চলবে	কম হলে model converge করার আগেই থেমে যেতে পারে।
tol (default=1e-3)	Improvement threshold	Training কখন থামবে তা নির্ধারণ করে।
early_stopping (default=False)	Validation score উন্নতি না হলে training বন্ধ করবে কিনা	Overfitting কমাতে সাহায্য করে।
validation_fraction (default=0.1)	Validation set-এর জন্য ডেটার অংশ	early_stopping=True হলে গুরুত্বপূর্ণ।
shuffle (default=True)	প্রতি epoch-এর আগে data shuffle হবে কিনা	সাধারণত True রাখা উচিত।
random_state	Random seed	Reproducibility-এর জন্য।
average (default=False)	Weight averaging ব্যবহার করবে কিনা	Noisy data-তে stability বাড়াতে পারে।

warm_start (default=False)	আগের training থেকে continue করবে কিনা	Incremental learning-এর জন্য উপকারী।
-----------------------------------	---------------------------------------	--------------------------------------

SGDRegressor-এর সবচেয়ে গুরুত্বপূর্ণ Parameter

Priority	Parameter	কারণ
☆☆☆	alpha	Regularization strength
☆☆☆	penalty	কোন regularization হবে
☆☆☆	learning_rate	Training stability
☆☆☆	eta0	Initial learning rate
☆☆	max_iter	Convergence নিশ্চিত করে
☆☆	early_stopping	Overfitting কমায়

বিষয়	মন্তব্য
Feature Scaling	SGDRegressor ব্যবহারের আগে অবশ্যই StandardScaler ব্যবহার করতে হবে
Learning Rate	ভুল learning rate হলে divergence বা poor convergence হতে পারে
Regularization	alpha ও penalty ঠিকভাবে tune করা অত্যন্ত গুরুত্বপূর্ণ

Logistic Regression

লজিস্টিক রিগ্রেশন মূলত একটি **ক্লাসিফিকেশন অ্যালগরিদম** (Classification Algorithm), যা বাইনারি ক্লাস (যেমন: 0 অথবা 1, হ্যাঁ অথবা না) প্রেডিক্ট করতে ব্যবহৃত হয়। লিনিয়ার রিগ্রেশন যেখানে $(-\infty$ থেকে $\infty)$ পর্যন্ত যেকোনো মান দিতে পারে, লজিস্টিক রিগ্রেশন সেখানে আউটপুটকে 0 থেকে 1 এর মধ্যকার একটি **প্রোবাবিলিটি** (Probability) বা সম্ভাবনায় রূপান্তর করে।

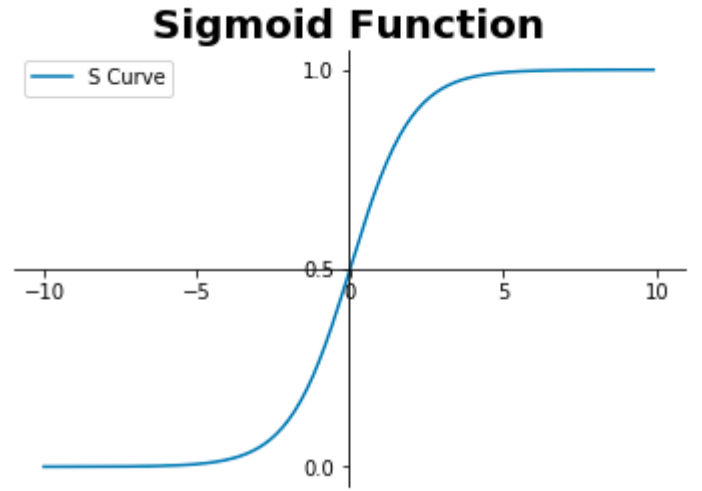
১. মূল সমীকরণ ও সিগময়েড ফাংশন (Core Equation & Sigmoid)

প্রথমে linear combination হিসাব করা হয়:

$$z = XW + b$$

এই z -কে sigmoid function-এর মাধ্যমে probability-তে রূপান্তর করা হয়:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Final prediction:

$$\hat{y} = \sigma(XW + b) = \frac{1}{1 + e^{-(XW+b)}}$$

Sigmoid-এর গুরুত্বপূর্ণ বৈশিষ্ট্য:

$$\begin{aligned} z = 0 & \quad \text{তখন } \sigma(z) = 0.5 \\ z \rightarrow +\infty & \quad \text{তখন } \sigma(z) \rightarrow 1 \\ z \rightarrow -\infty & \quad \text{তখন } \sigma(z) \rightarrow 0 \end{aligned}$$

Odds $\rightarrow$ Sigmoid Function (Full Derivation)

১. Odds কী?

ধরি কোনো event-এর probability হলো p

তাহলে:

$$\text{odds} = \frac{p}{1 - p}$$

মানে: success vs failure ratio

২. Log-Odds (Logit)

Odds-এর log নিলে:

$$\log(\text{odds}) = \log\left(\frac{p}{1-p}\right)$$

৩. Logistic Regression-এর assumption

$$z = \log\left(\frac{p}{1-p}\right)$$

অর্থাৎ:

- $z = \log\text{-odds}$

৪. Log undo করা (Exponential step)

আমরা জানি:

$$z = \log(\text{odds})$$

দুই পাশে exponential নিলে:

$$e^z = \text{odds}$$

৫. Odds বসানো

$$\frac{p}{1-p} = e^z$$

৬. এখন solve করি probability

$$\begin{aligned} p &= e^z(1-p) \\ p &= e^z - pe^z \\ p + pe^z &= e^z \\ p(1 + e^z) &= e^z \end{aligned}$$

৭. Final probability form

$$p = \frac{e^z}{1 + e^z}$$

৮. Sigmoid form এ রূপান্তর

উপরের expression কে rearrange করলে:

$$p = \frac{1}{1 + e^{-z}}$$

৯. Final Result (Sigmoid Function)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

২. কস্ট ফাংশন (Binary Cross-Entropy / Log Loss)

Logistic Regression-এ Mean Squared Error ব্যবহার করা হয় না, কারণ এটি non-convex সমস্যা তৈরি করে।

তাই ব্যবহার করা হয়:

$$J(W, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Loss Intuition

যখন $y = 1$:

$$\text{Loss} = -\log(\hat{y})$$

- $\hat{y} \approx 1 \rightarrow \text{loss} \approx 0$
- $\hat{y} \approx 0 \rightarrow \text{loss} \rightarrow \infty$

যখন $y = 0$:

$$\text{Loss} = -\log(1 - \hat{y})$$

- $\hat{y} \approx 0 \rightarrow \text{loss} \approx 0$
- $\hat{y} \approx 1 \rightarrow \text{loss} \rightarrow \infty$

৩. গ্রাডিয়েন্ট (Partial Derivatives)

Log loss + sigmoid combine করলে খুব সুন্দরভাবে gradient simplify হয়।

Linear part:

$$z = XW + b$$

$$\hat{y} = \sigma(z)$$

Weight gradient:

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

Bias gradient:

$$\frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Important Note:

এখানে:

- $\hat{y}$ = sigmoid output
- error = $\hat{y} - y$

৪. প্যারামিটার আপডেট (Gradient Descent)

$$W = W - \alpha \cdot \frac{\partial J}{\partial W}$$

$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

ব্যাকএন্ড ওয়ার্কফ্লো (Full Process)

Step 1: Initialization

- $W = 0$ বা ছোট random value
- $b = 0$

Step 2: Linear computation

$$z = XW + b$$

Step 3: Activation (Sigmoid)

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

Step 4: Loss computation

$$J(W, b)$$

Step 5: Gradient computation

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

Step 6: Update parameters

$$W, b \text{ update}$$

Step 7: Prediction rule

$$\begin{aligned}\hat{y} &\geq 0.5 \Rightarrow 1 \\ \hat{y} &< 0.5 \Rightarrow 0\end{aligned}$$

Python Code

```
1 import numpy as np
2
3 X = np.array([
4     [0.5, 1.5],
5     [1.0, 1.0],
6     [1.5, 0.5],
7     [3.0, 3.5],
8     [2.5, 4.5],
9     [4.0, 3.0]
10 ])
11
12 y = np.array([0, 0, 0, 1, 1, 1])
13
14 def sigmoid(z):
15     return 1 / (1 + np.exp(-z))
16
17 def predict_proba(X, W, b):
18     return sigmoid(np.dot(X, W) + b)
19
20 def predict(X, W, b, threshold=0.5):
21     return (predict_proba(X, W, b) >= threshold).astype(int)
22
23 def compute_cost(X, y, W, b):
24     y_pred = predict_proba(X, W, b)
25     return -np.mean(y*np.log(y_pred) + (1-y)*np.log(1-y_pred))
```

```
27 def compute_gradient(X, y, W, b):
28     error = predict_proba(X, W, b) - y
29     dW = (1/len(X)) * np.dot(X.T, error)
30     db = (1/len(X)) * np.sum(error)
31     return dW, db
32
33 def fit(X, y, lr, iters):
34     W = np.zeros(X.shape[1])
35     b = 0.0
36
37     for i in range(iters):
38         dW, db = compute_gradient(X, y, W, b)
39         W -= lr * dW
40         b -= lr * db
41
42     return W, b
43
44 W, b = fit(X, y, 0.1, 5000)
45
46 print(W, b)
47 print(predict(X, W, b))
```


Using Scikit Learn

```
1 from sklearn.linear_model import LogisticRegression, SGDClassifier
2 from sklearn.preprocessing import StandardScaler
3
4 lr = LogisticRegression(penalty=None)
5 lr.fit(X, y)
6
7 print(lr.coef_, lr.intercept_)
8 print(lr.predict(X))
9
10 scaler = StandardScaler()
11 X_scaled = scaler.fit_transform(X)
12
13 sgd = SGDClassifier(loss="log_loss", max_iter=5000, eta0=0.1, learning_rate
    ="constant")
14 sgd.fit(X_scaled, y)
15
16 print(sgd.coef_, sgd.intercept_)
17 print(sgd.predict(X_scaled))
```

LogisticRegression (sklearn.linear_model.LogisticRegression)

Logistic Regression classification সমস্যার জন্য ব্যবহৃত হয় এবং regularization-এর উপর অনেক বেশি নির্ভরশীল।

Parameter	অর্থ	কখন গুরুত্বপূর্ণ
C (default=1.0)	Regularization strength-এর inverse	সবচেয়ে গুরুত্বপূর্ণ parameter।
penalty (default='l2')	Regularization type (l1, l2, elasticnet, None)	Overfitting control ও feature selection-এর জন্য।
solver (default='lbfgs')	Optimization algorithm	Penalty-এর উপর নির্ভর করে solver নির্বাচন করতে হয়।
class_weight (default=None)	Imbalanced dataset handle করে	Imbalanced classification-এ খুব গুরুত্বপূর্ণ।
max_iter (default=100)	Optimization iteration সংখ্যা	Convergence warning এড়াতে বাড়ানো লাগে।
fit_intercept (default=True)	Bias term যুক্ত হবে কিনা	সাধারণত True রাখা উচিত।
l1_ratio	ElasticNet-এ L1/L2 ratio	শুধুমাত্র ElasticNet-এর জন্য।
multi_class	Multiclass strategy	Multiclass classification-এর ক্ষেত্রে গুরুত্বপূর্ণ।
tol (default=1e-4)	Convergence threshold	Training stopping criteria নির্ধারণ করে।
random_state	Random seed	Consistent result-এর জন্য।
warm_start (default=False)	আগের solution থেকে training continue করবে কিনা	Re-training দ্রুত করতে পারে।
n_jobs	Parallel processing	Speed বাড়ায়, model quality পরিবর্তন করে না।

Solver & Penalty Compatibility

Solver	L1	L2	ElasticNet	None
lbfgs	✗	✓	✗	✓
newton-cg	✗	✓	✗	✓
sag	✗	✓	✗	✓
liblinear	✓	✓	✗	✗
saga	✓	✓	✓	✓

LogisticRegression-এর সবচেয়ে গুরুত্বপূর্ণ Parameter

Priority	Parameter	কারণ
★ ★ ★	C	Regularization strength
★ ★ ★	penalty	Overfitting control
★ ★ ★	solver	Optimization compatibility
★ ★ ★	class_weight	Imbalanced dataset handling
★ ★	max_iter	Proper convergence
★ ★	l1_ratio	ElasticNet tuning

বিষয়	মন্তব্য
Feature Scaling	Logistic Regression-এর জন্যও StandardScaler ব্যবহার করা ভালো
Imbalanced Data	class_weight='balanced' অনেক ক্ষেত্রে performance বাড়ায়
Feature Selection	penalty='l1' coefficient শূন্য করে feature selection করতে পারে
Large Dataset	বড় dataset-এর জন্য saga solver সাধারণত ভালো কাজ করে

K-Nearest Neighbors (KNN)

1. Introduction to K-Nearest Neighbors

K-Nearest Neighbors (KNN) হলো Machine Learning-এর একটি জনপ্রিয় Supervised Learning Algorithm, যা Classification এবং Regression উভয় ধরনের সমস্যার সমাধান করতে ব্যবহৃত হয়। এটি একটি Non-Parametric এবং Lazy Learning Algorithm। অন্যান্য অনেক Machine Learning Algorithm-এর মতো KNN Training Phase-এ কোনো Mathematical Model তৈরি করে না। বরং সম্পূর্ণ Training Dataset মেমোরিতে সংরক্ষণ করে রাখে এবং Prediction করার সময় নতুন Data Point-এর সবচেয়ে কাছের প্রতিবেশীদের (Neighbors) খুঁজে বের করে সিদ্ধান্ত নেয়।

KNN-এর মূল ধারণা হলো:

Similar data points tend to exist close to each other in feature space.

অর্থাৎ, যেসব Data Point-এর বৈশিষ্ট্য একরকম, তারা সাধারণত Feature Space-এ কাছাকাছি অবস্থান করে।

2. Why Do We Need KNN?

বাস্তব জীবনে অনেক সমস্যা আছে যেখানে আমাদের একটি নতুন Observation-এর Category বা Value নির্ধারণ করতে হয়।

উদাহরণস্বরূপ:

- Email Spam নাকি Not Spam?
- Customer Product কিনবে নাকি কিনবে না?
- একজন রোগী রোগে আক্রান্ত নাকি সুস্থ?
- একটি বাড়ির সম্ভাব্য মূল্য কত?

এই ধরনের সমস্যার সমাধানে KNN ব্যবহার করা যায়।

KNN-এর বিশেষত্ব হলো এটি Data Distribution সম্পর্কে আগে থেকে কোনো Assumption করে না। ফলে জটিল Non-Linear Data-এর ক্ষেত্রেও এটি অনেক সময় ভালো কাজ করে।

3. Types of Problems Solved by KNN

Classification

Classification সমস্যায় Target Variable একটি Category হয়।

উদাহরণ:

Input	Output
Patient Data	Disease / No Disease
Email	Spam / Not Spam
Student Info	Pass / Fail

Classification-এর ক্ষেত্রে KNN Majority Voting ব্যবহার করে।

Regression

Regression সমস্যায় Target Variable Continuous Numeric Value হয়।

উদাহরণ:

Input	Output
House Features	House Price
Weather Data	Temperature
Employee Data	Salary

Regression-এর ক্ষেত্রে KNN Neighbors-এর Mean বা Median ব্যবহার করে।

4. Working Principle of KNN

KNN-এর সম্পূর্ণ কাজ Distance-এর উপর নির্ভরশীল।

ধরা যাক একটি নতুন Data Point এসেছে।

KNN নিচের ধাপগুলো অনুসরণ করে:

Step 1

একটি K Value নির্বাচন করা হয়।

$$K = 3$$

Step 2

নতুন Data Point থেকে Training Dataset-এর প্রতিটি Observation-এর Distance বের করা হয়।

Step 3

Distance অনুযায়ী ছোট থেকে বড় ক্রমে Sort করা হয়।

Step 4

সবচেয়ে কাছের K সংখ্যক Neighbor নির্বাচন করা হয়।

Step 5

Classification হলে Majority Voting এবং Regression হলে Average নেওয়া হয়।

5. Mathematical Foundation of KNN

KNN-এর সম্পূর্ণ Decision Making Distance Metrics-এর উপর ভিত্তি করে তৈরি।

দুটি Data Point-এর মধ্যকার Distance যত কম হবে, তারা তত বেশি Similar বলে বিবেচিত হবে।

5.1 Euclidean Distance

এটি KNN-এর সবচেয়ে ব্যবহৃত Distance Metric।

জ্যামিতিতে দুটি বিন্দুর সরলরেখার দূরত্ব নির্ণয়ের জন্য এটি ব্যবহার করা হয়।

যদি দুটি Point হয়:

$$P = (x_1, y_1)$$

এবং

$$Q = (x_2, y_2)$$

তাহলে তাদের Euclidean Distance:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

N-Dimensional Form

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

এই Formula-ই Scikit-Learn-এর KNN-এর Default Distance Metric।

5.2 Manhattan Distance

Euclidean Distance সরলরেখার দূরত্ব মাপে।

কিন্তু Manhattan Distance Grid Path ধরে Distance মাপে।

Formula:

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

High Dimensional Dataset-এর ক্ষেত্রে অনেক সময় Manhattan Distance Euclidean Distance-এর চেয়ে ভালো ফলাফল দেয়।

5.3 Minkowski Distance

Euclidean এবং Manhattan Distance আসলে Minkowski Distance-এর Special Case।

$$d(x, y) = (\sum_{i=1}^n |x_i - y_i|^p)^{1/p}$$

যেখানে,

$$p = 1$$

হলে Manhattan Distance

এবং

$$p = 2$$

হলে Euclidean Distance পাওয়া যায়।

6. Complete Numerical Example

ধরা যাক নিচের Dataset দেওয়া আছে।

Customer	Age	Income	Buy
C1	2	4	No
C2	4	6	Yes
C3	4	2	No
C4	6	8	Yes

এখন নতুন Customer:

$(3, 5)$

এবং

$K = 3$

প্রথম Customer-এর সাথে Distance:

$$d_1 = \sqrt{(3 - 2)^2 + (5 - 4)^2} = 1.41$$

দ্বিতীয় Customer-এর সাথে:

$$d_2 = 1.41$$

তৃতীয় Customer-এর সাথে:

$$d_3 = 3.16$$

চতুর্থ Customer-এর সাথে:

$$d_4 = 4.24$$

Distance অনুযায়ী Sort করলে:

Customer	Distance	Class
C1	1.41	No
C2	1.41	Yes
C3	3.16	No
C4	4.24	Yes

Nearest 3 Neighbor:

- No
- Yes
- No

Majority Vote:

$No = 2$
 $Yes = 1$

Final Prediction:

No

7. Choosing the Optimal K Value

KNN-এর Performance-এর সবচেয়ে গুরুত্বপূর্ণ অংশ হলো K নির্বাচন করা।

K খুব ছোট হলে

$$K = 1$$

Model Noise Follow করতে শুরু করে।

ফলাফল:

- High Variance
- Overfitting

K খুব বড় হলে

$$K = 100$$

Model Local Pattern ভুলে যায়।

ফলাফল:

- High Bias
- Underfitting

সাধারণভাবে:

$$K = \sqrt{N}$$

দিয়ে শুরু করা হয়।

যেখানে N হলো Training Sample-এর সংখ্যা।

8. Feature Scaling in KNN

KNN Distance-Based Algorithm হওয়ার কারণে Feature Scaling অত্যন্ত গুরুত্বপূর্ণ।

ধরা যাক:

Feature	Range
Age	0-100
Salary	0-1,000,000

Distance Calculation-এর সময় Salary পুরো Distance-কে Dominant করে ফেলবে।

ফলে Age-এর Effect প্রায় হারিয়ে যাবে।

এই সমস্যার সমাধানের জন্য:

Standardization

$$z = \frac{x - \mu}{\sigma}$$

Min-Max Scaling

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

প্রয়োগ করা হয়।

9. Curse of Dimensionality

KNN-এর সবচেয়ে বড় সীমাবদ্ধতাগুলোর একটি হলো Curse of Dimensionality।

Feature সংখ্যা বাড়তে থাকলে Data Point-গুলোর মধ্যে Distance-এর পার্থক্য কমতে শুরু করে।

ফলে:

- Nearest Neighbor খুঁজে পাওয়া কঠিন হয়
- Prediction Accuracy কমে যায়
- Computation Cost বৃদ্ধি পায়

এই কারণে High-Dimensional Dataset-এ KNN অনেক সময় দুর্বল পারফর্ম করে।

10. Computational Complexity

Operation	Complexity
Training	$O(1)$
Memory	$O(N)$
Prediction	$O(ND)$

যেখানে,

- N = Number of Samples
- D = Number of Features

এই কারণেই KNN-এর Training Fast হলেও Prediction তুলনামূলক Slow হয়।

KNN ব্যবহার করার সময় নিচের Parameters সবচেয়ে গুরুত্বপূর্ণ।

Parameter	Description
n_neighbors	K-এর মান নির্ধারণ করে
weights	uniform অথবা distance
metric	Distance Metric নির্বাচন করে
p	Minkowski Distance-এর Power
algorithm	auto, kd_tree, ball_tree, brute
leaf_size	KD Tree এবং Ball Tree Optimization
n_jobs	Parallel Processing

KNeighborsClassifier(

n_neighbors=5,

weights='distance',

metric='minkowski',

p=2,

algorithm='auto'

)

```

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

wine = load_wine()

X = wine.data
y = wine.target

print("Shape of X:", X.shape)
print("Shape of y:", y.shape)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

```

knn_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier(
        n_neighbors=5,
        weights="uniform",
        metric="minkowski",
        p=2
    ))
])

knn_pipeline.fit(X_train, y_train)
y_pred = knn_pipeline.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:")
print(accuracy)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

```

GridSearchCV in KNN

1. Why Do We Need GridSearchCV?

KNN-এর সবচেয়ে গুরুত্বপূর্ণ Hyperparameter হলো:

$$K = n_neighbors$$

কিন্তু প্রশ্ন হলো:

K কত নেব?

3?

5?

7?

11?

21?

যদি ভুল K নির্বাচন করি তাহলে Model-এর Performance অনেক কমে যেতে পারে।

ধরো:

K	Accuracy
3	88%
5	92%
7	95%
9	93%
11	90%

এখানে দেখা যাচ্ছে:

$$K = 7$$

সবচেয়ে ভালো।

কিন্তু Dataset দেখে আগে থেকে এটা জানা সম্ভব না।

তাই আমাদের বিভিন্ন K পরীক্ষা করতে হবে।

এই কাজটিই GridSearchCV স্বয়ংক্রিয়ভাবে করে।

2. What is GridSearchCV?

GridSearchCV হলো Scikit-Learn-এর একটি Hyperparameter Tuning Technique।

এটি বিভিন্ন Parameter Combination পরীক্ষা করে এবং Best Combination খুঁজে বের করে।

ধরো তুমি KNN-এর জন্য নিচের Parameters পরীক্ষা করতে চাও:

```
n_neighbors = [3,5,7,9]
weights = ['uniform','distance']
```

তাহলে মোট Combination হবে:

n_neighbors	weights
3	uniform
3	distance
5	uniform
5	distance
7	uniform
7	distance
9	uniform
9	distance

মোট:

$$4 \times 2 = 8$$

টি Model Train হবে।

3. Why Is It Called Grid Search?

ধরো Parameter Space হলো:

Parameter	Values
K	3,5,7
Weight	uniform,distance
p	1,2

তাহলে Grid তৈরি হবে:

K	Weight	p
3	uniform	1
3	uniform	2
3	distance	1
3	distance	2
5	uniform	1
5	uniform	2
5	distance	1
5	distance	2
7	uniform	1
7	uniform	2
7	distance	1
7	distance	2

মোট:

$$3 \times 2 \times 2 = 12$$

Combination I

GridSearchCV এই পুরো Grid Search করে।

তাই নাম:

Grid Search

4. What Does CV Mean?

CV = Cross Validation

Grid Search শুধু Train/Test Split ব্যবহার করে না।

বরং Cross Validation ব্যবহার করে।

5. Cross Validation Intuition

ধরো Dataset-এ:

1000

Samples আছে।

আমরা

$cv = 5$

দিলাম।

তাহলে Dataset 5 ভাগে ভাগ হবে।

Fold-1

Fold-2

Fold-3

Fold-4

Fold-5

Iteration 1:

Train:

2 3 4 5

Validation:

1

Iteration 2:

Train:

1 3 4 5

Validation:

2

Iteration 3:

Train:

1 2 4 5

Validation:

3

Iteration 4:

Train:

1 2 3 5

Validation:

4

Iteration 5:

Train:

1 2 3 4

Validation:

5

এরপর Average Accuracy বের করা হয়।

$$CV\ Score = \frac{Score_1 + Score_2 + Score_3 + Score_4 + Score_5}{5}$$

6. Backend Working of GridSearchCV

ধরি:

n_neighbors=[3,5,7]

weights=['uniform','distance']

Combination:

(3,uniform)

(3,distance)

(5,uniform)

(5,distance)

(7,uniform)

(7,distance)

মোট:

6

Combination I

এবং

cv=5

তাহলে:

$$6 \times 5 = 30$$

বার Model Train হবে।

GridSearchCV:

1. First Combination নেয়
2. 5 Fold CV চালায়
3. Average Score বের করে
4. Next Combination নেয়
5. Repeat করে

শেষে Highest Score যেটা দেয় সেটাকে Best Model ঘোষণা করে।

7. Important KNN Hyperparameters

GridSearchCV সাধারণত KNN-এর নিচের Parameters Tune করে।

n_neighbors

সবচেয়ে গুরুত্বপূর্ণ Parameter।

n_neighbors = [3,5,7,9,11]

ছোট K

→ Overfitting

বড় K

→ Underfitting

weights

Uniform

সব Neighbor-এর Vote সমান।

weights='uniform'

Distance

কাছের Neighbor-এর Vote বেশি।

weights='distance'

Weight:

$$w = \frac{1}{d}$$

p

Minkowski Distance Power।

p = 1

Manhattan

Distance

p = 2

Euclidean

Distance

metric

`metric='minkowski'`

Default |

অন্যান্য:

euclidean

manhattan

cosine

chebyshev

8. Example of Full Search Space

```
param_grid = {  
    "model__n_neighbors":[  
        3,5,7,9,11,15  
    ],  
    "model__weights":[  
        "uniform",  
        "distance"  
    ],  
    "model__p":[  
        1,  
        2  
    ]  
}
```

Total Models:

$$6 \times 2 \times 2 = 24$$

If

`cv=5`

Then

$$24 \times 5 = 120$$

Model Fits

9. Important Outputs

After Training:

Best Parameters

`grid.best_params_`

Output:

```
{  
'n_neighbors':7,  
'weights':'distance',  
'p':2  
}
```

Best Score

`grid.best_score_`

Output:

0.972

Meaning:

97.2%

Cross Validation Accuracy

Best Model

`grid.best_estimator_`

Returns:

Best Trained Pipeline

10. Advantages of GridSearchCV

Automatic Hyperparameter Tuning

নিজে নিজে Combination পরীক্ষা করে।

Uses Cross Validation

Result বেশি Reliable।

Finds Optimal Model

Best Parameters খুঁজে দেয়।

Easy to Use

Scikit-Learn-এ কয়েক লাইনের Code।

11. Disadvantages

Slow

Combination বেশি হলে সময় বেশি লাগে।

Computationally Expensive

Large Dataset-এ Costly।

Exhaustive Search

সব Combination পরীক্ষা করে।

কখনও কখনও অপ্রয়োজনীয়ও হতে পারে।

```

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

wine = load_wine()

X = wine.data
y = wine.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier())
])

```

```

param_grid = {
    "model__n_neighbors": [3,5,7,9,11,15],
    "model__weights": [
        "uniform",
        "distance"
    ],
    "model__p": [
        1,
        2
    ]
}

grid = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid.fit(X_train, y_train)

print("Best Parameters:")
print(grid.best_params_)

```

```
print("Best Parameters:")
print(grid.best_params_)

print("\nBest CV Score:")
print(grid.best_score_)

print("\nBest Pipeline:")
print(grid.best_estimator_)

test_accuracy = grid.score(X_test, y_test)

print("\nTest Accuracy:")
print(test_accuracy)
```